Stochastic-DTS: Risk-Aware Differentiable Tree Search for Stochastic Environments

1 Observed Phenomenon

Model-based Reinforcement Learning (MBRL) methods that use online planning, such as Differentiable Tree Search (DTS), achieve strong performance by jointly optimizing a world model and a search algorithm. However, DTS and similar methods are designed for deterministic worlds. When deployed in non-deterministic environments, their deterministic world models fail to capture environmental stochasticity, leading them to learn an "average" world dynamic. This can result in suboptimal or unsafe policies, as the planner is blind to low-probability, high-impact events.

2 Problem Statement

State-of-the-art differentiable search methods in MBRL are constrained by their reliance on deterministic world models, limiting their effectiveness in real-world scenarios that are inherently stochastic. While methods exist for learning stochastic world models (e.g., STORM), they are typically trained separately from the planner, which prevents the planner and model from co-adapting. This separation means the world model is not optimized for the specific states and transitions that are most critical for the planner's decision-making process.

Condition: We lack a framework that can jointly optimize a powerful, **stochastic** world model (capable of representing a distribution over next states) with a **differentiable tree search** algorithm in an end-to-end fashion.

Consequence: The benefits of differentiable planning and learning a world model that is specifically useful for the planner are lost in stochastic environments. Agents cannot reason effectively about uncertainty during planning, leading to poor sample efficiency and brittle policies that fail to generalize to novel situations or handle risk appropriately.

3 Motivation

Our proposed method, Stochastic Differentiable Tree Search (S-DTS), aims to bridge this gap.

- **Limitation of DTS:** Deterministic DTS transition functions are fundamentally mismatched with stochastic environments. They cannot represent or plan for multiple possible outcomes of an action.
- **Limitation of Separate Training:** Simply plugging a pre-trained stochastic world model (like STORM) into a planner misses the key insight of DTS. The world model would be trained to minimize global prediction error, which is different from being accurate in the specific, task-relevant regions of the state space that the planner explores.
- **The S-DTS Advantage:** S-DTS integrates **Variational Stochastic World Model (VS-WM)**, inspired by STORM's architecture, directly into the DTS framework. The core motivation is that **joint optimization provides a powerful, targeted learning signal**. The overall planning loss,

L_Q, backpropagates through the entire differentiable search process, including the sampled transitions from the VS-WM. This teaches the VS-WM not just to be a good general predictor, but to be an accurate predictor *specifically for the transitions that matter most for making good decisions.* The planner, in turn, learns to account for the specific uncertainties expressed by its co-adapted world model. This synergy is expected to yield more robust and efficient learning.

4 Hypothesis

By integrating a stochastic world model into the Differentiable Tree Search (DTS) framework and jointly optimizing all components end-to-end—including the world model parameters, the value function, and the search policy—the resulting agent will achieve significantly superior performance, risk-adjusted returns, and generalization in non-deterministic environments compared to (a) the original DTS with a deterministic model, (b) DTS with an ensemble of deterministic models, and (c) methods using a separately trained stochastic world model with MCTS.

5 Proposed Method

S-DTS extends the DTS architecture by replacing its deterministic transition function with a full Stochastic World Model and adapting the search and backup mechanisms accordingly.

1. **Variational Stochastic World Model (VS-WM)**
   - **Encoder** $q_\phi(z_t|o_t)$: A CNN that acts as an inference network, mapping a real observation o_t to the parameters of a posterior distribution over latent states z_t. We use categorical distributions as in STORM.
   - **Decoder** $p_\phi(\hat{o}_t|z_t)$: A deconvolutional network that reconstructs the observation $\hat{o}_t$ from a latent state z_t.
   - **Dynamics Predictor** $g_\phi(P(\hat{z}_{t+1})|h_t)$: A GPT-like Transformer that takes a sequence of latent-action embeddings and outputs the parameters of the *prior* distribution $P(\hat{z}_{t+1})$ over the next latent state. This is the predictive model used by the planner.
2. **Online Search in Latent Space** The search process follows DTS but is adapted for the VS-WM.
   a. **Expansion Phase**: The best-first search, guided by the stochastic tree expansion policy π_θ, selects a node N^* to expand. For each action a:
      - The dynamics predictor g_ϕ is used to get the prior distribution over the next latent state: $P(\hat{z}_{\text{child}}|h_{N^*}, a)$.
      - A next latent state $\hat{z}_{\text{child}}$ is *sampled* from this distribution. To enable gradient flow, this sampling is made differentiable using the *Gumbel-Softmax trick* for the categorical latent variables.
      - The new node is added to the tree.

   b. **Backup Phase (Expected Backups)**: Value backups must account for transition stochasticity. The Q-value for a node N and action a is

$$Q(N, a) = R_\theta(h_N, a) + \gamma\, \mathbb{E}_{\hat{z}_{\text{child}} \sim P(\hat{z}_{\text{child}}|h_N, a)} \big[V(\hat{z}_{\text{child}}) \big].$$

   We approximate this expectation via *Monte Carlo estimation* by drawing K samples from the dynamics predictor:

$$Q(N, a) \approx R_\theta(h_N, a) + \gamma\, \frac{1}{K} \sum_{k=1}^{K} V\big(\hat{z}_{\text{child},k}\big).$$

   Finally, the node's value is

$$V(N) = \max_a Q(N, a).$$

3. **Gradient Flow and Variance Reduction** We have two sources of stochasticity, each requiring careful gradient management:

   - **Gradient through World Model**: The Gumbel-Softmax trick allows the end-to-end planning loss L_Q to flow back through the sampling step in the expansion phase, providing a direct optimization signal to the Dynamics Predictor g_ϕ.
   - **Gradient for Search Policy**: The search policy π_θ is trained via REINFORCE. To combat high variance, we use the **telescoping sum trick** from DTS (Paper 2, Section 3.5) and add a learned value function as a baseline.

4. **Training and Loss Functions** The entire S-DTS model is trained end-to-end on a composite loss:

$$L_{\text{total}} = \lambda_Q L_Q + \lambda_D L_D + L_{\text{WM}}$$

   - L_Q: The primary RL loss (e.g., MSE between search output $Q_\theta(s, a \mid \tau)$ and target Q-values).
   - L_D: The Conservative Q-Learning (CQL) loss from DTS (Paper 2, Eq. 10), applied to the final $Q_\theta(s, a \mid \tau)$ values to regularize the policy in offline settings.
   - L_{WM}: The world model loss, adapted from STORM (Paper 1, Eq. 3), ensures the learned latent space is well-structured: $L_{\text{WM}} = L_{\text{rec}} + \beta_1 L_{\text{dyn}} + \beta_2 L_{\text{rep}}$
     - L_{rec}: Reconstruction loss ($\|o - \hat{o}\|^2$) from the decoder.
     - L_{dyn}: Dynamics loss $\text{KL}\big[\text{sg}(q_\phi) \parallel g_\phi\big]$ to train the predictor.
     - L_{rep}: Representation loss $\text{KL}\big[q_\phi \parallel \text{sg}(g_\phi)\big]$ to regularize the encoder.

6 Experimental Outline

The experimental plan is structured in two phases to manage technical risk.

1. **Phase 1: Feasibility and Scaling Analysis**

   - **Environment**: Custom stochastic grid worlds (e.g., '"slippery ice") where stochasticity can be precisely controlled.
   - **Model**: A "Tiny S-DTS" with a small (e.g., 2-layer, as in STORM) Transformer and limited tree trials.
   - **Goal**:
     - Verify the stability of the end-to-end training pipeline.
     - Benchmark computational cost. Primary Metric: Plot wall-clock time vs. number of tree trials T, MC samples K, Transformer depth.
     - Compare variance reduction techniques (telescoping sum vs. baselines).

2. **Phase 2: Full Benchmark Evaluation**

   - **Environments**:
     - Stochastic Grid World with rare catastrophic events.
     - Stochastic Procgen: Introduce probabilistic elements (e.g., enemy movement, action outcomes) to a subset of Procgen games ('climber', 'ninja').
   - **Models for Comparison**:
     - S-DTS (Proposed)
     - DTS with Ensembles (Baseline 1): The original DTS using an ensemble of 5 deterministic transition models to capture uncertainty.

  - DTS-Deterministic (Baseline 2): The original DTS from Paper 2.
  - MCTS + Separately Trained VS-WM (Baseline 3): Train our VS-WM using only L_{WM}, then use it with a standard MCTS planner.
  - Model-Free Baseline (e.g., CQL): A strong model-free baseline.
- **Primary and Secondary Success Criteria and Metrics**:
  - Performance: Average cumulative reward.
  - Risk-Sensitivity: Conditional Value at Risk (CVaR) on the catastrophic grid world to measure performance in the worst-case outcomes.
  - Generalization: Train on one level of stochasticity (e.g., 20% slip) and test on unseen levels (10%, 40%) to plot degradation curves.
  - Uncertainty Calibration: Track the KL-divergence between the VS-WM's predicted transition distribution and the empirical distribution of next states on a held-out dataset.
  - Sample Efficiency: Performance as a function of training steps/data.

7 Concrete Example

Consider the "slippery ice" grid world from the original idea, where move_north can result in slipping left or right. A pit with a large negative reward is nearby.

- **DTS with Ensembles (Baseline)**: An ensemble of deterministic models might each predict a different, but fixed, outcome. The planner might average these paths or see variance in predictions, but it doesn't reason about the probability of each outcome. It might still choose a path close to the pit if the average path seems safe.
- **S-DTS (Proposed)**: The VS-WM predicts a single distribution over next states, $P(z') = \{\, 0.7 : \text{north},\ 0.15 : \text{slip-left},\ 0.15 : \text{slip-right}\}$. The expected backup rule $\mathbb{E}[V(z')]$ explicitly incorporates the low-probability, high-cost outcome of slipping into the pit. The $Q\big(N, \text{move\_north}\big)$ value will be lower if the slip outcomes lead to the pit. Consequently, the S-DTS planner will learn to select a safer path that is further from the pit, even if it is slightly longer, demonstrating superior risk-awareness.

8 Potential Pitfalls & Mitigations

- **Risk 1 – Training Instability:** The interaction of REINFORCE, Gumbel-Softmax sampling, and VAE losses can be highly volatile.
  - **Fallback:** Our phased approach allows us to debug this on simpler environments first. We will use robust variance reduction (telescoping sum + baseline), gradient clipping, and entropy regularization for both the search policy and the world model's output distribution to prevent premature collapse.
- **Risk 2 – Computational Overhead:** The triple loop of training steps, search trials, and MC samples for backups with a Transformer model is very expensive.
  - **Fallback:** Our feasibility analysis (Phase 1) is designed to find a workable trade-off. We will start with an efficient Transformer (as in STORM) and small T and K values, scaling up only as the budget allows.
- **Risk 3 – Inaccurate World Model:** The VS-WM might fail to learn the true stochastic dynamics, especially rare events.
  - **Fallback:** We can adjust the weights of the loss terms (e.g., increase β_1 on L_{dyn} or the weight of L_Q. Curriculum learning, starting with deterministic and gradually increasing stochasticity, can guide the model. The joint optimization with the L_Q signal is our main tool to force the model to be accurate where it matters.

References

[1] Zhang, W., Wang, G., Sun, J., Yuan, Y., & Huang, G. (2023). *STORM: Efficient Stochastic Transformer-based World Models for Reinforcement Learning*. arXiv:2310.09615.

[2] Mittal, D., & Lee, W. S. (2024). *Differentiable Tree Search in Latent State Space*. arXiv:2401.11660.